

AI ENGINEER ROADMAP

Curso Personalizado de AI Engineering

André Rizzato

Projeto base: DistributedOrderSystem

Volume 1 - Semana 1

ÍNDICE

Setup de Ambiente	3
Fundamentos de LLM	4
Anthropic API na Prática	5
Integração com o Projeto	6
Glossário	8
Referência de Arquitetura	10
Embeddings e Similaridade Semântica	11
RAG do Zero - Chunking e Retrieval	13
RAG do Zero - Augmentation e Generation	14
Ollama Local e GPU	16
Conteúdo Programático	18

Setup de Ambiente

Antes de escrever qualquer linha de IA, preparamos o terreno. A camada de IA do DistributedOrderSystem vive dentro de uma pasta **ai/** separada da solução .NET, para não misturar responsabilidades: o C# continua sendo a camada de domínio, o Python entra como camada de inteligência.

Passo 1 - Ambiente Python

Confirme a versão instalada e instale as duas dependências iniciais do curso: o SDK oficial da Anthropic e o utilitário para carregar variáveis de ambiente.

```
python --version
pip install anthropic python-dotenv
```

Comandos de terminal - Semana 1

Passo 2 - Estrutura de pastas

```
DistributedOrderSystem/
  ai/
    .env                <- ANTHROPIC_API_KEY (no .gitignore)
    .gitignore
  week1/
    first_call.py       <- exercicio atual
    streaming_call.py    <- proximo passo
```

Estrutura de arquivos da Semana 1

Passo 3 - Chave de API e segurança

A chave é obtida em **console.anthropic.com** → **API Keys** → **Create Key**. Ela nunca vai para o Git: o arquivo **.env** guarda a chave localmente e o **.gitignore** impede que ela seja versionada.

```
# .env
ANTHROPIC_API_KEY=sua_chave_aqui

# .gitignore
.env
__pycache__/
*.pyc
```

Conteúdo de .env e .gitignore

Fundamentos de LLM

Um **LLM** (Large Language Model) é um modelo estatístico treinado para prever a próxima unidade de texto (token) dado o contexto anterior. Isso é suficiente, em escala, para gerar texto coerente, responder perguntas, escrever código e raciocinar sobre problemas.

Tokens e janela de contexto

Um **token** é a unidade mínima de texto que o modelo processa - em média, algo entre 3 e 4 caracteres em português. A **context window** é o número máximo de tokens (entrada + saída) que o modelo consegue considerar em uma única chamada. Todo o histórico de conversa, o system prompt e a pergunta atual competem pelo mesmo espaço.

System prompt vs. user prompt

O **system prompt** define o papel, as regras e o tom do assistente antes da conversa começar - é onde configuramos, por exemplo, que o assistente deve responder sempre em português e conhecer o contexto do DistributedOrderSystem. O **user prompt** é a mensagem específica do usuário em cada turno.

Temperature

Controla a aleatoriedade da geração: valores baixos (perto de 0) tornam a resposta mais determinística e previsível - ideal para tarefas de negócio como classificar um pedido. Valores altos aumentam a criatividade/diversidade, mas também o risco de respostas menos precisas.

Prévia: embeddings

Além de gerar texto, um modelo pode transformar texto em um vetor numérico (**embedding**) que representa seu significado. Dois textos com significados próximos geram vetores próximos no espaço. Essa ideia é a base do RAG, que construiremos do zero ainda na Fase 1.

Anthropic API na Prática

Toda interação com o modelo passa pelo endpoint **messages.create**. Os parâmetros centrais são:

- **model** - qual modelo será usado (ex: claude-sonnet-4-6)
- **max_tokens** - limite de tokens de saída
- **system** - o system prompt (papel do assistente)
- **messages** - lista de turnos da conversa (role + content)

Exercício: first_call.py

Este é o primeiro script que André executa no curso - uma chamada simples, com system prompt já ancorado no contexto do DistributedOrderSystem.

```
# week1/first_call.py
import os
from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

client = Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    system="""Voce e um assistente especializado no sistema DistributedOrderSystem.
    Esse sistema gerencia pedidos distribuidos com microservicos em .NET/C#.
    Responda sempre em portugues.""",
    messages=[
        {"role": "user", "content": "O que e um sistema de pedidos distribuido e
        quais sao os principais desafios?"}
    ]
)

print(response.content[0].text)
print(f"\n--- Uso de tokens ---")
print(f"Input:  {response.usage.input_tokens}")
print(f"Output: {response.usage.output_tokens}")
```

Código completo - week1/first_call.py

O que observar na resposta

- O modelo respondeu dentro do contexto do system prompt?
- Quantos tokens de input foram usados - o system prompt já custa tokens
- A latência - tempo até a primeira palavra aparecer

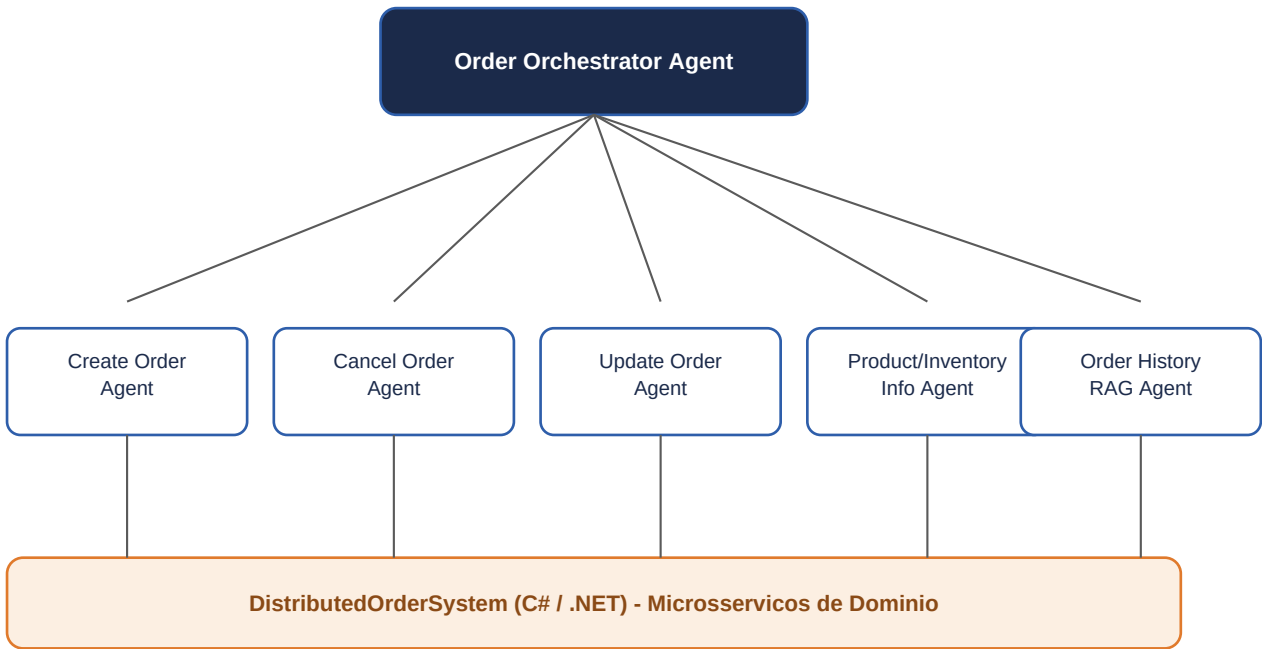
Integração com o Projeto

O DistributedOrderSystem não é um projeto de exemplo descartável - é o backbone técnico de uma futura empresa. A decisão arquitetural central: **Python assume a camada de IA** (agentes, RAG, orquestração) e **C#/.NET permanece intocado como camada de domínio** (regras de negócio, persistência, APIs).

Mapeamento de agentes para o domínio de pedidos

Agente genérico	Agente no projeto
Orchestrator	Order Orchestrator Agent
Booking	Create Order Agent
Cancellation	Cancel Order Agent
Reschedule	Update Order Agent
Information	Product/Inventory Info Agent
RAG/FAQ	Order History RAG Agent

Diagrama multi-agente aplicado ao domínio



Fluxo: usuario -> Orchestrator -> agente especialista -> API .NET do dominio

Figura 3.1 - O Orchestrator recebe a intenção do usuário, roteia para o agente especialista correspondente, que por sua vez chama a API .NET do domínio.

Glossário

LLM (Large Language Model)

Modelo estatístico treinado para prever o próximo token de um texto, usado para gerar respostas, código e raciocínio.

Token

Unidade mínima de texto processada pelo modelo; aproximadamente 3-4 caracteres em português.

Embedding

Representação numérica (vetor) do significado de um texto, usada para medir similaridade semântica.

Similaridade de cosseno

Medida do ângulo entre dois vetores (ignora magnitude); padrão para comparar embeddings de texto.

Chunking

Divisão de um texto longo em pedaços menores (chunks) antes de gerar embeddings, para permitir retrieval granular.

Overlap (chunking)

Trecho de texto repetido entre chunks consecutivos, para não perder contexto na fronteira do corte.

Retrieval

Etapa do RAG que busca, por similaridade vetorial, os chunks mais relevantes para uma pergunta.

Augmentation

Etapa do RAG que injeta os chunks recuperados como contexto explícito dentro do prompt enviado ao LLM.

Grounding

Instruir o LLM a responder apenas com base no contexto fornecido, reduzindo o risco de alucinação.

RAG (Retrieval-Augmented Generation)

Técnica que busca informação relevante em uma base de conhecimento antes de gerar a resposta do modelo.

Agente

Sistema que usa um LLM para decidir ações, chamar ferramentas e iterar até cumprir um objetivo.

Tool Use / Function Calling

Capacidade do modelo de chamar funções externas (ex: consultar um pedido no .NET) durante a conversa.

MCP (Model Context Protocol)

Protocolo aberto para conectar modelos a ferramentas e fontes de dados de forma padronizada.

Orchestrator

Agente responsável por rotear a intenção do usuário para o agente especialista correto.

Fine-tuning

Processo de re-treinar um modelo pré-existente com dados específicos de um domínio.

LoRA / QLoRA

Técnicas eficientes de fine-tuning que ajustam poucos parâmetros extras, reduzindo custo computacional.

Prompt Engineering

Prática de estruturar instruções para obter o melhor comportamento possível do modelo.

Context Window

Quantidade máxima de tokens (entrada + saída) que o modelo consegue processar em uma chamada.

Temperature

Parâmetro que controla a aleatoriedade/criatividade da geração de texto.

System Prompt

Instrução inicial que define papel, tom e regras do assistente antes da conversa do usuário.

Referência de Arquitetura

Visão consolidada do plano de 26 semanas, dividido em 6 fases. Cada fase entrega capacidades concretas dentro do DistributedOrderSystem, sem abstrair prematuramente para um template genérico.

Fase	Semanas	Foco
1 - Fundamentos	1-4	Python, API Anthropic, embeddings, RAG do zero
2 - Agentes e MCP	5-10	Ollama, tool use, MCP server, Semantic Kernel, LangGraph
3 - RAG Avançado	11-16	Hybrid search, reranking, LlamaIndex, RAGAS, DSPy
4 - Fine-tuning	17-22	LoRA/QLoRA com Unsloth, DPO, vLLM, MLOps
5 - Produção + Negócio	23-26	LiteLLM, extração do backbone, instanciação do nicho

Stack consolidada

Camada	Tecnologia	Papel
IA / Orquestração	Python (LangGraph, LlamaIndex, FastMCP, Unsloth, RAGAS, DSPy)	Camada principal de IA
Domínio / APIs	C# / .NET (microserviços existentes)	Camada de domínio - não mexer
Bridge opcional	Semantic Kernel (.NET)	Ponte .NET <-> Python apenas
LLM local	Ollama (Llama 3.2 / Phi-3.5)	Dev offline, zero custo
LLM cloud	Anthropic API (claude-sonnet-4-6)	Produção
Gateway	LiteLLM	Roteamento cloud/local
Observabilidade	LangSmith / Langfuse	Traces e métricas

Regra anti-abstração prematura: construir o concreto primeiro (Orders, ponta a ponta). Generalizar para o template core/ + packs/ só ao instanciar o segundo domínio, na Fase 5.

Embeddings e Similaridade Semântica

Um **embedding** é um vetor numérico que representa o significado de um texto. Textos com significados próximos geram vetores próximos no espaço vetorial. A Anthropic não tem modelo nativo de embeddings - o curso usa a **Voyage AI** (**voyage-4-large** para indexação em nuvem, **voyage-4-nano** open-weight para rodar local).

Similaridade de cosseno - a fórmula

Dado dois vetores A e B, a similaridade de cosseno é:

$$\cos(A, B) = (A \cdot B) / (|A| \cdot |B|)$$

$A \cdot B$ = produto escalar = soma de ($a_i \cdot b_i$) termo a termo
 $|A|$ = norma de A = raiz quadrada da soma de a_i ao quadrado
 $|B|$ = norma de B = raiz quadrada da soma de b_i ao quadrado

O numerador mede o quanto os vetores apontam na mesma direção; o denominador normaliza pelo tamanho de cada vetor, deixando o resultado só sobre a direção (o resultado fica sempre entre -1 e 1).

Cosseno vs. distância euclidiana - exemplo numérico

Vetor A = (1, 1) e vetor B = (2, 2). B é literalmente A esticado - mesma direção, magnitude diferente.

Distância euclidiana:

$$\text{raiz}((2-1)^2 + (2-1)^2) = \text{raiz}(2) = 1.41 \quad \rightarrow \text{parece "diferente"}$$

Similaridade de cosseno:

$$(1 \cdot 2 + 1 \cdot 2) / (\text{raiz}(2) \cdot \text{raiz}(8)) = 4 / 4 = 1.0 \quad \rightarrow \text{direção idêntica}$$

Em embeddings de texto, a magnitude do vetor costuma refletir coisas como tamanho do texto, não o significado. Cosseno ignora isso e compara só a direção - por isso é o padrão em NLP, e não a distância euclidiana.

Aprofundamento: quatro perguntas em aberto (em linguagem simples)

1. Por que cosseno, e não distância euclidiana?

Porque distância euclidiana também é afetada pelo **tamanho** do vetor, não só pela direção - e o tamanho do embedding costuma refletir coisas como tamanho do texto, não o significado. Dois textos com o mesmo sentido mas tamanhos diferentes podem parecer "distantes" na euclidiana e "idênticos" no cosseno (veja o exemplo numérico acima: A e B apontam pro mesmo lugar, cosseno = 1.0, mas a distância euclidiana entre eles não é zero). Cosseno olha só pra direção, por isso é o padrão em NLP.

2. Negação e o estado Cancelled

Embeddings não entendem negação como lógica - entendem como padrão de palavras que aparecem juntas. "Pedido foi cancelado" e "pedido não foi cancelado" usam quase as mesmas palavras, no mesmo contexto, e por isso ficam próximos no espaço vetorial mesmo significando o oposto. Como **Cancelled** é estado terminal da máquina de estados do pedido (Pending -> Confirmed -> Shipped -> Delivered / Cancelled), um fato categórico e exato como esse deve vir de

filtro estruturado no banco de dados, não de busca semântica - busca semântica responde "sobre o que é o texto", não "isso é verdadeiro ou falso".

3. Cosseno negativo ($\cos = -1$)

$\cos = 1$ significa mesma direção (significado quase idêntico); $\cos = 0$ significa direções perpendiculares (sem relação); $\cos = -1$ significaria direções opostas (significado oposto). Na prática, com embeddings de texto, valores próximos de -1 quase não aparecem: textos com significados opostos costumam cair perto de 0 (sem relação), não perto de -1. Esses modelos são treinados para agrupar textos parecidos, não para codificar "oposto lógico" - o espaço vetorial não tem um conceito forte de antônimo.

4. Códigos de erro internos (ex.: `ORD_TIMEOUT_502`)

O modelo de embedding não sabe que isso é um identificador interno do sistema - ele quebra o código em pedaços (`ORD`, `TIMEOUT`, `502`) e gera um vetor a partir de associações genéricas aprendidas no treino ("`502`" puxa semântica de erro HTTP, "`TIMEOUT`" puxa semântica genérica de tempo esgotado), sem saber o que o código significa especificamente no domínio. Por isso não se indexa o código cru - indexa-se a descrição em linguagem natural (ex.: "`ORD_TIMEOUT_502`: tempo excedido aguardando confirmação do provedor de pagamento").

RAG do Zero - Chunking e Retrieval

Antes de usar um framework (LangChain, LlamaIndex), o curso constrói um RAG mínimo à mão, para entender o mecanismo por baixo do capô: dividir texto em pedaços (**chunking**), transformar cada pedaço em embedding, e recuperar os mais relevantes para uma pergunta (**retrieval**).

Chunking de tamanho fixo com overlap

A janela desliza sobre o texto em passos menores que o próprio tamanho do chunk, criando sobreposição entre chunks vizinhos:

$\text{passo} = \text{tamanho_chunk} - \text{overlap}$

```
Exemplo: tamanho_chunk=60, overlap=15 -> passo = 45
posicao:  0      45      90      135     151
texto:   |-----|-----|-----|-----|
chunk 0:  [0.....60)
chunk 1:   [45.....105)
chunk 2:    [90.....150)
chunk 3:     [135.....151)
```

O overlap evita que uma informação seja perdida bem na fronteira do corte - mas não impede o corte de cair no meio de uma palavra, como aconteceu aqui ("Motivo" virou "...o:" no início do chunk 1).

Exercício: rag_chunking_retrieval.py

```
def chunk_text(texto, tamanho_chunk=60, overlap=15):
    chunks = []
    passo = tamanho_chunk - overlap
    inicio = 0
    while inicio < len(texto):
        chunks.append(texto[inicio:inicio + tamanho_chunk])
        inicio += passo
    return chunks
```

Código - week3/rag_chunking_retrieval.py (trecho)

Ao embeddar a pergunta, o input_type usado é "query" (não "document") - a Voyage gera embeddings ligeiramente diferentes para query vs. documento, uma assimetria intencional do modelo, não um bug.

Resultado real (nota de suporte do pedido 4521)

Pergunta: "o cliente pediu reembolso?"

```
[0.5493] 'o: item chegou danificado. Cliente pediu dinheiro de volta v'
[0.4521] 'eiro de volta via PIX. Reembolso processado em 3 dias uteis.'
```

O chunk vencedor não contém a palavra "reembolso" - contém "Cliente pediu dinheiro de volta", com a mesma estrutura sujeito+verbo+objeto da pergunta. O chunk com a palavra literal "Reembolso" fica em segundo porque fala do **status** do reembolso (prazo de processamento), não de quem pediu o quê - **relevância tópica** (mesma área do assunto) é diferente de **relevância proposicional** (responder à mesma pergunta), e é a segunda que o embedding está de fato capturando.

RAG do Zero - Augmentation e Generation

Chunking e retrieval (Capítulo 07) respondem "quais pedaços de texto são relevantes". Faltam os dois últimos passos do RAG: transformar esses chunks em resposta em linguagem natural.

```
pergunta -> embed -> retrieve (top-k chunks)
      |
      v
    AUGMENTATION: monta um prompt injetando
                  os chunks como contexto explícito
      |
      v
    GENERATION: LLM responde usando
                APENAS esse contexto
```

Augmentation

É literalmente concatenar os chunks recuperados dentro do prompt que vai pro LLM - "aumentar" a pergunta do usuário com informação que ele não tinha antes.

Generation com grounding

No **system prompt**, o modelo é instruído a responder **só** com base no contexto fornecido, e a dizer que não sabe se a resposta não estiver lá. Sem essa instrução, o LLM pode completar a resposta com conhecimento geral dele - que pode estar certo por acaso, mas quebra a garantia de que a resposta vem dos dados reais do pedido, não da memória do modelo. Essa técnica se chama **grounding**.

```
def generate_answer(pergunta, chunks_recuperados):
    contexto = "\n".join(f"- {texto}" for _, texto in chunks_recuperados)

    system = (
        "Voce responde perguntas sobre pedidos do DistributedOrderSystem "
        "usando APENAS o contexto fornecido. Se a resposta nao estiver "
        "no contexto, diga que nao ha informacao suficiente."
    )

    response = anthropic_client.messages.create(
        model="claude-sonnet-4-6", max_tokens=300, system=system,
        messages=[{"role": "user", "content": f"Contexto:\n{contexto}\n\nPergunta: {pergunta}"}],
    )
    return response.content[0].text
```

Código - week4/rag_generation.py (trecho)

Resultado real - pipeline ponta a ponta

Pergunta: "o cliente pediu reembolso"

Chunks recuperados (contexto que vai para o LLM):

```
[0.5819] 'o: item chegou danificado. Cliente pediu dinheiro de volta v'
[0.4726] 'eiro de volta via PIX. Reembolso processado em 3 dias uteis.'
```

Resposta gerada pelo LLM:

Com base no contexto fornecido, sim - o cliente relatou que o item chegou danificado e solicitou reembolso via PIX. O reembolso foi processado com

prazo de 3 dias uteis.

Resposta corretamente **grounded**: só usa o que estava nos 2 chunks recuperados, nada inventado. Detalhe fino: os scores de cosseno ficaram levemente diferentes do Capítulo 07 ($0.5493 \rightarrow 0.5819$, $0.4521 \rightarrow 0.4726$) porque a pergunta desta vez não tinha "?" no final - o embedding representa o texto exato enviado, pontuação inclusa. A ordem do ranking não mudou, só o valor do score - lembrete de que "similaridade semântica" ainda é sensível a variações superficiais do input.

Ollama Local e GPU

Fase 2 começa trocando a API cloud por um LLM rodando localmente, via **Ollama** num container Docker - zero custo por chamada, funciona offline, argumento de privacidade para clientes PME sensíveis a dado.

docker-compose.yml

```
services:
  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
```

Docker não repassa GPU pra nenhum container por padrão - é opt-in explícito via **deploy.resources.reservations.devices**, não detecção automática. Sem esse bloco, o Ollama roda em CPU mesmo com uma GPU disponível no host.

CPU vs. GPU - medido, não estimado

Mesma pergunta, mesmo modelo (llama3.2), antes e depois de adicionar a reserva de GPU no compose:

	CPU (antes)	GPU (depois)
Geracao	~13-14 tok/s	~95 tok/s
Tempo total (1a call)	~44-47s	~6-7s
Tempo total (call quente) -		~0.7s

~7-8x mais rápido na geração com a GPU. O **load_duration** (carregar o modelo na VRAM) só acontece uma vez por reinício do container - depois disso fica quente e cai pra perto de zero, igual acontecia na RAM da CPU.

ollama_first_call.py - mesmo padrão da Semana 1

```
OLLAMA_URL = "http://localhost:11434/api/chat"

def chat(pergunta, system=""):
    payload = {
        "model": "llama3.2",
        "messages": [
            {"role": "system", "content": system},
            {"role": "user", "content": pergunta},
        ],
        "stream": False,
    }
    return requests.post(OLLAMA_URL, json=payload, timeout=120).json()
```


O endpoint `/api/chat` do Ollama aceita o mesmo formato `messages=[{role, content}]` da API da Anthropic - dá pra comparar direto com o `first_call.py` da Semana 1. Diferença: Ollama expõe métricas de timing (`load_duration`, `eval_count`) que a API cloud não expõe.

Conteúdo Programático

As 26 semanas do curso, semana a semana, com o status de cada item no momento desta geração do ebook.

Fase	Semana	Item	Status
1 - Fundamentos	Sem 1	Python profissional, API Anthropic	Concluída
1 - Fundamentos	Sem 2	Embeddings e similaridade semântica (Voyage AI)	Concluída
1 - Fundamentos	Sem 3-4	RAG do zero: chunking, retrieval, augmentation e generation ponta a ponta	Concluída
2 - Agentes e MCP	Sem 5	Ollama local + docker-compose + GPU passthrough	Concluída
2 - Agentes e MCP	Sem 6-7	Tool use, loop ReAct, primeiro MCP server	Pendente
2 - Agentes e MCP	Sem 8	Intent classification (few-shot + Pydantic/Instructor)	Pendente
2 - Agentes e MCP	Sem 9	Semantic Kernel como ponte .NET <-> Python	Pendente
2 - Agentes e MCP	Sem 10	LangGraph básico, multi-agent orchestrator-worker	Pendente
3 - RAG Avançado	Sem 11-16	Hybrid search, reranking, LlamaIndex, RAGAS, DSPy, Qdrant	Pendente
4 - Fine-tuning	Sem 17-22	LoRA/QLoRA (Unsloth), DPO, vLLM, MLOps, ML clássico	Pendente
5 - Produção + Negócio	Sem 23-26	LiteLLM, extração do backbone, instanciação do nicho	Pendente

- **Concluída** - exercício rodado e verificado com o André.
- **Em andamento** - parte do trabalho da semana já entregue.
- **Pendente** - ainda não iniciada.